

Національний технічний університет України «Київський
політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Дискретна математика

Лабораторна робота №3

«Графи. Способи представлення графів. Остовні дерева. Пошук
найкоротших шляхів»

Виконав:
студент групи ІО-41
Давидчук А. М.
Залікова книжка № 4106

Перевірів
Пономаренко А. М.

Тема: «Графи. Способи представлення графів. Основні дерева. Пошук найкоротших шляхів».

Мета: Вивчення властивостей графів, способів їх представлення та основних алгоритмів на графах.

Короткі теоретичні відомості:

Граф – структура, що складається з множини вершин (точок) та ребер (з'єднань між ними).

Орієнтований граф – граф, у якому кожне ребро має напрям (дуга).

Ациклічний граф (DAG) – орієнтований граф, що не містить циклів.

Матриця суміжності – спосіб подання графа у вигляді квадратної таблиці, де значення в клітинці $[i][j]$ вказує на наявність і вагу ребра з вершини i до j .

Вага ребра – числове значення, яке може означати відстань, вартість, час переходу тощо.

Найкоротший шлях – послідовність вершин, у якій сума ваг ребер є мінімальною серед усіх можливих шляхів між заданими вершинами.

Топологічне сортування – впорядкування вершин орієнтованого ациклічного графа так, що кожна вершина з'являється **раніше**, ніж усі вершини, до яких з неї веде ребро.

Завдання за варіантом $4106 \bmod 10 + 1 = 6 + 1 = 7$:

1. Вивчити принципи застосування алгоритму топологічного сортування до пошуку найкоротших шляхів у графі. Написати програму пошуку найкоротших шляхів у графі з застосуванням алгоритму топологічного сортування.
2. За правилом, наданим викладачем, сформулювати матрицю ваг $C = [c_{ij}]$ графа.
3. Представити граф, заданий матрицею ваг C , у графічній формі та виділити найкоротший шлях між заданими викладачем вершинами, сформований за алгоритмом топологічного сортування.

Код:

```
import tkinter as tk
from tkinter import filedialog, messagebox
import networkx as nx
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

def read_matrix_from_file(filename):
    with open(filename, 'r') as f:
        return [list(map(int, line.strip().split())) for line in f]

def shortest_paths_toposort(matrix, start):
    n = len(matrix)
    G = nx.DiGraph()
    for i in range(n):
```

```

        for j in range(n):
            if matrix[i][j] != 0 and matrix[i][j] != float('inf'):
                G.add_edge(i, j, weight=matrix[i][j])

    try:
        top_order = list(nx.topological_sort(G))
    except nx.NetworkXUnfeasible:
        raise ValueError("Граф містить цикл. Алгоритм працює тільки для
DAG.")

    dist = [float('inf')] * n
    dist[start] = 0
    path = [-1] * n

    for u in top_order:
        for v in G.successors(u):
            weight = G[u][v]['weight']
            if dist[v] > dist[u] + weight:
                dist[v] = dist[u] + weight
                path[v] = u

    return G, dist, path

def reconstruct_path(path, start, end):

    if path[end] == -1: return []
    result = [end]

    while result[-1] != start: result.append(path[result[-1]])

    return result[::-1]

def draw_graph(G, shortest_path):
    pos = nx.spring_layout(G)
    edge_labels = nx.get_edge_attributes(G, 'weight')
    nx.draw(G, pos, with_labels=True, node_color='lightblue',
node_size=600)
    nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels)
    path_edges = list(zip(shortest_path, shortest_path[1:]))
    nx.draw_networkx_edges(G, pos, edgelist=path_edges, edge_color='red',
width=3)

def run_toposort_from_input():
    try:
        matrix = eval(entry_matrix.get("1.0", "end"))
        start = int(entry_start.get())
        end = int(entry_end.get())
        G, dist, path = shortest_paths_toposort(matrix, start)
        path_seq = reconstruct_path(path, start, end)
        ax.clear()
        draw_graph(G, path_seq)
        canvas.draw()
        label_result.config(text=f"Найкоротша відстань: {dist[end]}\nШлях:
{path_seq}")
    except Exception as e:

```

```

        messagebox.showerror("Помилка", str(e))

def load_matrix_from_file():

    filename = filedialog.askopenfilename(filetypes=[("Text files",
        "*.txt")])
    if filename:
        try:
            matrix = read_matrix_from_file(filename)
            entry_matrix.delete("1.0", "end")
            entry_matrix.insert("1.0", str(matrix))
        except Exception as e:
            messagebox.showerror("Помилка", str(e))

# GUI
root = tk.Tk()
root.title("Найкоротший шлях (топологічне сортування)")

tk.Label(root, text="Матриця ваг (наприклад:
[[0,5,0],[0,0,3],[0,0,0]])").pack()
entry_matrix = tk.Text(root, height=5, width=40)
entry_matrix.pack()

# Додати підтримку Ctrl+C, Ctrl+V, Ctrl+X
entry_matrix.bind("<Control-c>", lambda e:
entry_matrix.event_generate("<<Copy>>"))
entry_matrix.bind("<Control-C>", lambda e:
entry_matrix.event_generate("<<Copy>>"))
entry_matrix.bind("<Control-v>", lambda e:
entry_matrix.event_generate("<<Paste>>"))
entry_matrix.bind("<Control-V>", lambda e:
entry_matrix.event_generate("<<Paste>>"))
entry_matrix.bind("<Control-x>", lambda e:
entry_matrix.event_generate("<<Cut>>"))
entry_matrix.bind("<Control-X>", lambda e:
entry_matrix.event_generate("<<Cut>>"))

tk.Button(root, text="Завантажити 3 файлу",
command=load_matrix_from_file).pack()

frame_inputs = tk.Frame(root)
frame_inputs.pack()
tk.Label(frame_inputs, text="Початкова вершина: ").grid(row=0, column=0)
entry_start = tk.Entry(frame_inputs, width=5)
entry_start.grid(row=0, column=1)

tk.Label(frame_inputs, text="Кінцева вершина: ").grid(row=0, column=2)
entry_end = tk.Entry(frame_inputs, width=5)
entry_end.grid(row=0, column=3)

tk.Button(root, text="Знайти найкоротший шлях",
command=run_toposort_from_input).pack()

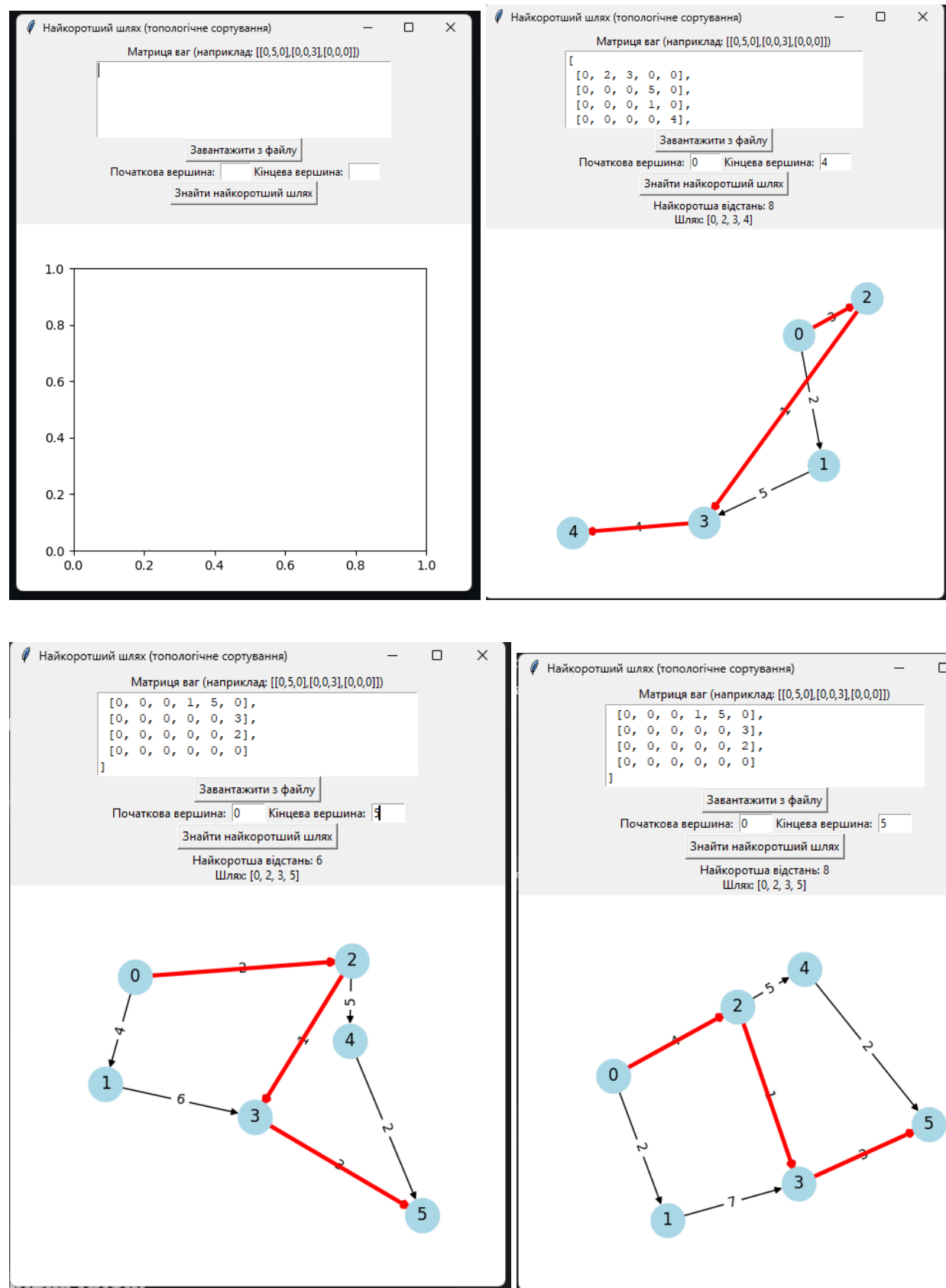
label_result = tk.Label(root, text="")
label_result.pack()

```

```
fig, ax = plt.subplots(figsize=(5, 4))
canvas = FigureCanvasTkAgg(fig, master=root)
canvas.get_tk_widget().pack()

root.mainloop()
```

Скріншоти результатів виконання:



Висновок: У цій лабораторній роботі було реалізовано пошук найкоротших шляхів у орієнтованому ациклічному графі за допомогою топологічного сортування. Сортування дозволило обробляти вершини в правильному порядку, що спростило обчислення мінімальних відстаней. Програма успішно знаходить найкоротший шлях, візуалізує граф та виділяє відповідні ребра. Робота показала практичну користь топологічного сортування для ефективної реалізації алгоритмів на орієнтованих ациклічних графах.